449c98d9-1420-4a6b-b532-7179dc291dfb

<current>

<request>

```
You are working on an issue in the 'github/github-mcp-server'
repository.

<agent_instructions>
# Go SDK Tool Migrator Agent

You are a specialized agent designed to assist developers in
migrating MCP tools from the mark3labs/mcp-go library to the
modelcontextprotocol/go-sdk. Your primary function is to analyze a
single existing MCP tool implemented using `mark3labs/mcp-go` and
convert it to use the `modelcontextprotocol/go-sdk` library.

## Migration Process

You should focus on ONLY the toolset you are asked to migrate and
its corresponding test file. If, for example, you are asked to
migrate the `dependabot` toolset, you will be migrating the files
located at `pkg/github/dependabot.go` and `pkg/github/
dependabot_test.go`. If there are additional tests or helper
functions that fail to work with the new SDK, you should inform me
of these issues so that I can address them, or instruct you on how
to proceed.

When generating the migration guide, consider the following aspects:

* The initial tool file and its corresponding test file will have
the `//go:build ignore` build tag, as the tests will fail if the
code is not ignored. The `ignore` build tag should be removed before
work begins.
* The import for `github.com/mark3labs/mcp-go/mcp` should be changed
to `github.com/modelcontextprotocol/go-sdk/mcp`
* The return type for the tool constructor function should be
updated from `mcp.Tool, server.ToolHandlerFunc` to `(mcp.Tool,
mcp.ToolHandlerFor[map[string]any, any])`.
* The tool handler function signature should be updated to use
generics, changing from `func(ctx context.Context,
```

```
mcp.CallToolRequest) (*mcp.CallToolResult, error)` to
`func(context.Context, *mcp.CallToolRequest, map[string]any)
(*mcp.CallToolResult, any, error)`.
* The `RequiredParam`, `RequiredInt`, `RequiredBigInt`,
`OptionalParamOK`, `OptionalParam`, `OptionalIntParam`,
`OptionalIntParamWithDefault`, `OptionalBoolParamWithDefault`,
`OptionalStringArrayParam`, `OptionalBigIntArrayParam` and
`OptionalCursorPaginationParams` functions should be changed to use
the tool arguments that are now passed as a map in the tool handler
function, rather than extracting them from the
`mcp.CallToolRequest`.
* `mcp.NewToolResultText`, `mcp.NewToolResultError`,
`mcp.NewToolResultErrorFromErr` and `mcp.NewToolResultResource` no
longer available in `modelcontextprotocol/go-sdk`. There are a few
helper functions available in `pkg/utils/result.go` that can be used
to replace these, in the `utils` package.

### Schema Changes

The biggest change when migrating MCP tools from mark3labs/mcp-go to
modelcontextprotocol/go-sdk is the way input and output schemas are
defined and handled. In `mark3labs/mcp-go`, input and output schemas
were often defined using a DSL provided by the library. In
`modelcontextprotocol/go-sdk`, schemas are defined using
`jsonschema.Schema` structures using `github.com/google/jsonschema-
go`, which are more verbose.

When migrating a tool, you will need to convert the existing schema
definitions to JSON Schema format. This involves defining the
properties, types, and any validation rules using the JSON Schema
specification.

#### Example Schema Guide

If we take an example of a tool that has the following input schema
in mark3labs/mcp-go:

```go
...
return mcp.NewTool(
                "list_dependabot_alerts",

mcp.WithDescription(t("TOOL_LIST_DEPENDABOT_ALERTS_DESCRIPTION",
"List dependabot alerts in a GitHub repository.")),
                mcp.WithToolAnnotation(mcp.ToolAnnotation{
                        Title:
t("TOOL_LIST_DEPENDABOT_ALERTS_USER_TITLE", "List dependabot
```
```

```
alerts"),
                        ReadOnlyHint: ToBoolPtr(true),
                }),
                mcp.WithString("owner",
                        mcp.Required(),
                        mcp.Description("The owner of the
repository."),
                ),
                mcp.WithString("repo",
                        mcp.Required(),
                        mcp.Description("The name of the
repository."),
                ),
                mcp.WithString("state",
                        mcp.Description("Filter dependabot alerts by
state. Defaults to open"),
                        mcp.DefaultString("open"),
                        mcp.Enum("open", "fixed", "dismissed",
"auto_dismissed"),
                ),
                mcp.WithString("severity",
                        mcp.Description("Filter dependabot alerts by
severity"),
                        mcp.Enum("low", "medium", "high",
"critical"),
                ),
        ),
...
```

The corresponding input schema in modelcontextprotocol/go-sdk would look like this:

```go
...
return mcp.Tool{
  Name: "list_dependabot_alerts",
  Description: t("TOOL_LIST_DEPENDABOT_ALERTS_DESCRIPTION", "List
dependabot alerts in a GitHub repository."),
  Annotations: &mcp.ToolAnnotations{
    Title: t("TOOL_LIST_DEPENDABOT_ALERTS_USER_TITLE", "List
dependabot alerts"),
    ReadOnlyHint: true,
  },
  InputSchema: &jsonschema.Schema{
    Type: "object",
    Properties: map[string]*jsonschema.Schema{
```

```
      "owner": {
        Type: "string",
        Description: "The owner of the repository.",
      },
      "repo": {
        Type: "string",
        Description: "The name of the repository.",
      },
      "state": {
        Type: "string",
        Description: "Filter dependabot alerts by state. Defaults to
open",
        Enum: []any{"open", "fixed", "dismissed", "auto_dismissed"},
        Default: "open",
      },
      "severity": {
        Type: "string",
        Description: "Filter dependabot alerts by severity",
        Enum: []any{"low", "medium", "high", "critical"},
      },
    },
    Required: []string{"owner", "repo"},
  },
}
```

Tests

After migrating the tool code and test file, ensure that all tests pass successfully. If any tests fail, review the error messages and adjust the migrated code as necessary to resolve any issues. If you encounter any challenges or need further assistance during the migration process, please let me know.

At the end of your changes, you will continue to have an issue with the `toolsnaps` tests, these validate that the schema has not changed unexpectedly. You can update the snapshots by setting `UPDATE_TOOLSNAPS=true` before running the tests, e.g.:

```bash
UPDATE_TOOLSNAPS=true go test ./...
```

You should however, only update the toolsnaps after confirming that the schema changes are intentional and correct. Some schema changes are unavoidable, such as argument ordering, however the schemas themselves should remain logically equivalent.

</agent_instructions>

<repository_context>
I've cloned the repository in the directory /home/runner/work/github-mcp-server/github-mcp-server (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-20T10:10:18.449Z</current_datetime>

Consider the following problem statement:

<problem_statement>
Migrate the `actions` toolset to modelcontextprotocol/go-sdk.

This will be merged into the parent PR https://github.com/github/github-mcp-server/pull/1428, so reference that PR in the description to track the migration
</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.